

NaTrack System Design

Scale estimation

Population. US adults 65 and older are about 54 million, and roughly 72 percent of adults 60 and older have hypertension, so the addressable group is about 38 million elderly hypertensive Americans. That is the ceiling, not the user base.

Design target. Assume 1 million active users, about 2.6 percent adoption, as the scaled design point. The short term pilot is 10,000. The pipeline is identical, only the numbers move.

Per user. Roughly 3 wet meals a day, about 30 bites a meal, so about 90 to 100 bite events per user per day. Four sensors fuse into one bite event on the device: EC salinity read through an ADS1115 for a clean signal, load cell weight read through an HX711, DS18B20 temperature, and MPU-6050 motion. The device also computes eating pace and drives a local LED cue, so pace feedback never touches the cloud. The cloud never sees four streams per spoon, it sees one summarized bite. This matters, see the deep dive.

Derived load at 1M users:

- Writes: 100M bite events per day, about 1,200 per second average, about 10,000 per second at meal peaks.
- Concurrent spoons recording: about 31,000 on average, about 100,000 to 150,000 at the dinner peak.
- Storage: a bite event is about 300 bytes, so about 30 GB per day, about 11 TB per year, archived to S3.
- Reads: far lighter, a few thousand clinicians loading time ranged dashboards during clinic hours, fully cacheable, plus a thin live session channel. Write heavy, read light.

Functional Requirements

- A device fuses four sensors into per bite events (salinity, weight, temperature, motion) and computes a sodium estimate as weight times salinity.
- The device gives the eater a real time pace cue on an LED, red, computed on device from the interval between bites.
- The system ingests, validates, and durably stores every bite per patient.
- A clinician views a patient's per bite, per meal, and multi day sodium trend against a target, plus eating pace metrics, average seconds between bites and quickest bite.
- A clinician sets a per patient sodium target and is flagged on upward drift.
- A patient logs their own health readings, blood pressure, weight, and notes, through a patient portal.

- A clinician can watch a live eating session in real time over a WebSocket, which also powers the demo.
- Out of scope for v (NOTE to flag: possible to integrate)1: potassium substitute alerts

Non-Functional Requirements

- Scale to about 1M users, 100M writes per day, 10k writes per second peak, 100k concurrent devices.
- Availability over strong consistency. A bite appearing a second late is fine, the dashboard must not go down. The live session channel is best effort, a dropped live bite is still durably stored.
- Durability is non negotiable. A logged bite is a health record and must never be lost.
- Low read latency for the dashboard, under a few hundred ms, and device store and forward when home Wi-Fi drops.
- Security and privacy. Per device certificate auth, encrypted in transit and at rest. The Ingest and Query Lambdas run in a private subnet with no internet route, reaching DynamoDB through a VPC gateway endpoint, guarded by a security group and NACL. KMS encrypts both tables at rest, with the tightest IAM on the Patients table, which holds the patient logged medical data. Clinicians are scoped to their patients through Cognito. Health data, so real deployment carries compliance obligations including an AWS BAA, not legal advice.

Core Entities

- **Patient:** patientId, sodium target, assigned clinician, condition.
- **Device:** deviceId, patientId, calibration, status.
- **BiteEvent:** patientId, timestamp, deviceId, salinityIndex, weightGrams, sodiumEstimate, tempC, biteIntervalSec, flags.
- **MealSummary:** derived aggregate, patientId, mealId, start, end, biteCount, totalSodium, avgBiteIntervalSec, minBiteIntervalSec, paceFlag.
- **HealthLog:** patient logged, patientId, timestamp, systolic, diastolic, weightKg, note.
- **Clinician:** userId, role, patient list.

API

Ingest, device side:

- MQTT publishes `devices/{deviceId}/bites` to IoT Core, per device cert auth.

Read and write, clinician and patient side, REST behind API Gateway and Cognito:

- `GET /v1/patients/{id}/bites?from=&to=` time ranged raw bites.

- `GET /v1/patients/{id}/summary?range=day|week|month` precomputed trend, sodium and pace.
- `GET /v1/patients?clinicianId=` patient list.
- `PUT /v1/patients/{id}/target` set sodium target.
- `POST /v1/patients/{id}/health-log` patient logs blood pressure, weight, or a note.
- `POST /v1/devices/{id}/pair` bind device to patient.

Live session, real time:

- WebSocket connect at `wss://.../session/{patientId}`, the ingest Lambda pushes each new bite to subscribed clinicians. Powers the live view and the demo.

Deep Dive

Edge aggregation is the core scaling decision. On the device the four sensors sample fast, roughly 70 raw readings per second, but only a fused bite event leaves the spoon. The fusion computes the sodium estimate, weight from the load cell times salinity from the EC probe, and the eating pace from the gap between bites, and it drives the LED locally so pace feedback has zero cloud latency. Without edge fusion, 100k spoons times 70 samples would be 7M messages per second and would bankrupt ingest. Sending bites, not samples, cuts that to about 10k per second at peak.

Storage and partitioning. Put BiteEvents in DynamoDB with partition key `patientId` and sort key timestamp. That co locates one patient's bites for the exact time range query the dashboard makes, and spreads writes evenly across a million partitions, so no hot partition. A GSI maps clinician to patients. TTL plus DynamoDB Streams archive bites older than 90 days to S3, queried later with Athena, keeping the hot table small. Patient medical data and the sodium target live in a separate Patients table with stricter IAM.

Ingest buffering. IoT Core to an IoT Rule to a queue, SQS or Kinesis, to a batch writing Lambda to DynamoDB. The queue absorbs the 10x meal time spike, protects the database, and gives at least once durability with retries. Idempotent writes keyed on `deviceId` plus timestamp dedupe replays.

Read path and precompute. DynamoDB Streams feed an aggregation Lambda that writes MealSummary rows including the pace metrics, so the dashboard reads small summaries, not thousands of raw bites, and caches them with DAX or CloudFront.

Real time path. After the ingest Lambda durably writes a bite, it also pushes it to the API Gateway WebSocket, which fans out to any clinician watching that patient's live session. This is best effort and additive, the durable store is the source of truth, so a missed live push never loses data. The same path drives the hackathon demo.

Frontend. A React single page app serves three views, the clinician dashboard, the patient portal for health logging, and the live demo. It is hosted on S3 behind CloudFront, or on Amplify, authenticated through Cognito, and calls the REST API for history and the WebSocket for live bites. The browser never touches DynamoDB directly, the Query API does.

Availability and consistency. Eventual consistency is acceptable, durability is not, so the queue guarantees delivery and the writes are idempotent. Nothing must run continuously, so serverless holds from 10k to 1M users with no rearchitecture.

Local demo note. The same API contract runs on a small local backend for a reliable hackathon demo, no venue network or cloud dependency, and deploys to AWS unchanged because the JSON bite event and the endpoints are identical.